Automaton Specification Language Documentation

Automaton Emulator

Contents

1 Introduction

The Automaton Specification Language (ASL) is a domain-specific language designed to define finite automata for simulation. It supports Deterministic Finite Automata (DFA), Non-deterministic Finite Automata (NFA), and Pushdown Automata (PDA).

2 File Structure

An ASL file is divided into sections, each denoted by a header in square brackets. Sections can appear in any order, though the following order is conventional:

1. `[alphabet]` (DFA/NFA) or `[alphabet_states]` and `[alphabet_stack]` (PDA)
2. `[states]`
3. `[initial_state]`
4. `[accept_states]`
5. `[transitions]`
6. `[settings]` (optional)

3 Sections

3.1 Alphabet

For DFA and NFA, the `[alphabet]` section defines the set of symbols accepted by the automaton.
For PDA, two sections are required:

- `[alphabet_states]`: The set of symbols that can appear in the input string.
- `[alphabet_stack]`: The set of symbols that can be pushed onto or popped from the stack.

Symbols are space-separated and can span multiple lines.

3.2 States

The `[states]` section lists all possible states in the automaton. Like the alphabet, states are space-separated.

3.2.1 State Ranges

To define a sequence of states with a common prefix and a numerical range, you can use the range syntax: `prefix[start-end]`.

```
q[0-15] # Expands to q0 q1 q2 ... q15
```

3.3 Initial State

The `[initial_state]` section must contain exactly one state from the defined states list. This represents the starting point of the simulation.

3.4 Accept States

The `[accept_states]` section lists the states that lead to an 'Accepted' result if the simulation ends in one of them. State ranges are also supported here.

3.5 Transitions

The `[transitions]` section defines the state transition function.

3.5.1 Wildcards

For DFA and NFA, the asterisk `*` can be used as a wildcard for input symbols, representing all symbols in the defined alphabet.

```
q0, * -> q1 # Transition to q1 on any symbol from the alphabet
```

In DFAs, a specific transition defined for a symbol will take precedence over a wildcard transition if it appears later in the file.

3.5.2 Epsilon Transitions

The ampersand `&` represents an epsilon transition, which occurs without consuming any input symbol. This is supported in NFA and PDA.

3.6 Settings

The `[settings]` section allows for optional behavioral modifications of the automaton. Settings are listed line-by-line or space-separated.

3.6.1 RemainInCurrentStateOnUndefinedTransition

When this setting is enabled, the automaton will not 'Hang' if it encounters a symbol for which no transition is defined for the current state. Instead, it will remain in its current state and continue processing the rest of the input string. This is particularly useful for building text-based games or parsers where some inputs should simply be ignored or keep the state unchanged.

4 Transition Syntax

4.1 DFA Transitions

For Deterministic Finite Automata, each transition must follow the strict format:

```
source_state, symbol -> next_state
```

4.2 NFA Transitions

Non-deterministic Finite Automata support additional syntax for flexibility:

Single Transition:

```
q0, a -> q1
```

Multiple Next States:

```
q0, a -> q1, q2
```

Multiple Input Symbols:

```
q0, (a, b) -> q1
```

Epsilon Transition:

```
q0, & -> q1
```

4.3 PDA Transitions

Pushdown Automata transitions include stack operations:

```
source_state, input_symbol, stack_pop -> next_state, stack_push
```

- `stack_pop`: The symbol that must be at the top of the stack to take this transition. Use `&` to pop nothing.
- `stack_push`: The symbol(s) to push onto the stack. Use `&` to push nothing. Multiple symbols can be pushed (space-separated).

5 Lexical Rules

- **Comments:** Any text following a `#` symbol on a line is ignored.
- **Whitespace:** Leading and trailing whitespace on lines is ignored. Empty lines are ignored.
- **Case Sensitivity:** Section headers are case-insensitive (`[ALPHABET]` is the same as `[alphabet]`).

6 Example

```
[alphabet]
a b

[states]
s0 s1

[initial_state]
s0

[accept_states]
s1

[transitions]
s0, a -> s1
s1, b -> s1
s1, a -> s0
```

Listing 1: Example DFA file

```
[alphabet]
0 1

[states]
q[0-2]

[initial_state]
q0

[accept_states]
q2

[transitions]
q0, * -> q0
q0, 1 -> q1
q1, (0, 1) -> q2
```

Listing 2: Example NFA file with ranges and wildcards